🎥 Google Meet Video Recording & Defense Teleprompter

Distributed Big Data Pipeline • Hadoop HDFS, Apache Spark ETL & Spark MLlib Analytics

HDFS → Spark → Spark MLlib

Teammate A (Driver / Storage Lead)

Shares screen, runs commands, presents Steps 1 & 2 (HDFS Data Lake) & Step 7 Wrap-Up

Teammate B (Spark ETL & Analytics Lead)

Presents Steps 3 & 4 (Distributed Spark ETL, CPCB 16h validity & 3 Diurnal/AQI Charts)

Teammate C (Spark MLlib ML Lead)

Presents Steps 5 & 6 (Chronological 24h Forecasting +16.6% & 6-Tier Hazard Confusion Matrix)

🎬 **VIDEO OPENING (30 SECONDS) • TEAMMATE A (SCREEN SHARER)**

*"Hello everyone! Welcome to our demonstration of our Big Data project on **Urban Air Quality Analytics and Forecasting**.*
*In this project, we built an end-to-end distributed system using **Hadoop HDFS** for storage, **Apache Spark** for distributed ETL, and **Spark MLlib** for machine learning at scale.*
*We're analyzing real continuous environmental data from three major Indian cities — **Delhi NCR**, **Bengaluru**, and **Hyderabad** — strictly adhering to official Central Pollution Control Board guidelines.*
To keep this walkthrough smooth and easy to follow, we've organized our run into numbered execution scripts. Let's start with Step 1."

🎬 Step 1: Inspecting Raw Telemetry in Hadoop HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ **TERMINAL ACTION (WHAT DRIVER RUNS)**

```
./1_hdfs_show_raw.sh
```

🖥️ **SCREEN OUTPUT (WHAT IS DISPLAYED)**

```
$ hdfs dfs -ls -h /air_quality/raw/
Found 1 items
-rw-r--r--   1 yashsonawane supergroup     47.3 M 2026-10-14 23:13 /air_quality/raw/raw_hourly_measurements.jsonl
```

🗣️ **WHAT TO SAY (NATURAL HUMAN FLOW)**

"Starting with Step 1, our screen driver is running `./1_hdfs_show_raw.sh`.

Right here on screen, you can see our raw telemetry stored directly inside Hadoop HDFS under `/air_quality/raw/`.

This file, **raw_hourly_measurements.jsonl**, is about **47 Megabytes** and holds **96,000 real hourly readings** collected via the OpenAQ API across Delhi, Bengaluru, and Hyderabad.

Because we stage this in HDFS, Hadoop automatically distributes it using standard 128 MB blocks with 3x replication, making our ingestion layer completely fault-tolerant.

Now, let's move to Step 2 to see our optimized storage layer."

Visual Pointers on Screen:

- Point to **47.3 M**: Real continuous telemetry (96,000 rows), not simulated data.
- Point to **/air_quality/raw/**: Demonstrates proper HDFS raw data staging.

🎬 Step 2: Inspecting Partitioned Parquet Lake in HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./2_hdfs_show_parquet.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ hdfs dfs -ls -h /air_quality/processed/hourly_aqi_parquet/
Found 4 items
-rw-r--r--   1 yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/_SUCCESS
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Bengaluru
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Delhi NCR
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Hyderabad
```

🗣️ WHAT TO SAY (NATURAL HUMAN FLOW)

"In Step 2, we run `./2_hdfs_show_parquet.sh` to inspect our processed data lake.

Notice that instead of slow, uncompressed CSVs, we converted everything into **columnar Apache Parquet with Snappy compression**, partitioned by city: Bengaluru, Delhi NCR, and Hyderabad.

This gives us a huge performance advantage called **Partition Pruning**. Whenever our downstream Spark queries ask for Delhi data, Spark reads only the Delhi directory and skips the other cities entirely, saving substantial disk I/O and query time.

Now, my teammate will walk you through our distributed Spark ETL pipeline."

Visual Pointers on Screen:

- Point to **_SUCCESS**: Official Hadoop output committer validation mark.
- Point to **city=XXX folders**: Hive-style partitioning for fast parallel scans.

🎬 Step 3: Running Distributed Apache Spark ETL & CPCB Math

Speaker: Teammate B (Spark ETL & Analytics Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./3_spark_run_etl.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/spark_etl.py
================ STARTING DISTRIBUTED SPARK ETL ================
Apache Spark Version: 4.2.0
[1/6] Ingesting raw dataset from: hdfs://localhost:9000/air_quality/raw/...
  Ingested 96,000 raw records.
[3/6] Normalizing timestamps to IST and harmonizing pollutant units...
[4/6] Enforcing CPCB completeness rule (>=16 valid hours/day)...
  Compliant station-days retained: 1,320
[5/6] Computing sub-indices and National Air Quality Index (NAQI)...
  Max sub-index applied across PM2.5, PM10, NO2, SO2, CO, O3.
[6/6] Writing partitioned Parquet dataset to HDFS...
ETL Pipeline execution finished in 20.48 seconds.
```

🗣️ WHAT TO SAY (NATURAL HUMAN FLOW)

"Moving to Step 3, we execute `./3_spark_run_etl.sh` to launch our distributed Apache Spark ETL job.

Watch how fast this runs across the cluster — in just about **20 seconds**, Spark processes all 96,000 records.

During this stage, Spark handles three critical operations: first, it converts all timestamps to Indian Standard Time and standardizes units; second, it enforces the official **CPCB 16-hour completeness rule** — requiring at least 16 valid readings a day so missing sensor hours don't distort our results; and third, it computes the exact non-linear piecewise formula to calculate the final National AQI.

Next in Step 4, let's explore the analytics and visual insights."

Visual Pointers on Screen:

- Point to **20.48 seconds**: Demonstrates high-speed distributed execution.
- Point to **>=16 valid hours/day**: Shows strict regulatory adherence.

Step 4: Distributed Spark Analytics & Summary Aggregations

Speaker: Teammate B (Spark ETL & Analytics Lead)

TERMINAL ACTION (WHAT DRIVER RUNS)

```
./4_spark_analytics.sh
```

SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/run_analytics.py
================ SPARK ANALYTICS & VISUALIZATION PIPELINE ================
Spark reading partitioned dataset from HDFS...
[Query 1] Computing 24-hour diurnal pollutant patterns...
[Query 2] Computing CPCB health category distribution across cities...
+-----------+---------+---------+---------+---------+---------+---------+
|city       |Good %   |Satisf. %|Moderate%|Poor %   |V.Poor % |Severe % |
+-----------+---------+---------+---------+---------+---------+---------+
|Bengaluru  |14.2%    |62.1%    |21.8%    |1.9%     |0.0%     |0.0%     |
|Delhi NCR  |0.0%     |2.4%     |8.1%     |12.5%    |14.5%    |62.5%    |
|Hyderabad  |8.5%     |48.2%    |35.1%    |7.2%     |1.0%     |0.0%     |
+-----------+---------+---------+---------+---------+---------+---------+
City Air Quality Overview:
  Delhi NCR  -> Avg PM2.5: 121.9 µg/m³ | Avg NAQI: 342.6 (Severe Smog)
  Bengaluru  -> Avg PM2.5:  28.8 µg/m³ | Avg NAQI:  78.4 (Satisfactory)
  Hyderabad  -> Avg PM2.5:  42.1 µg/m³ | Avg NAQI: 104.2 (Moderate)
```

WHAT TO SAY (NATURAL HUMAN FLOW)

"In Step 4, we run `./4_spark_analytics.sh`. Spark runs distributed SQL aggregations to calculate summary statistics and generates our high-resolution analytical plots.

Look at the summary table on screen:

In **Delhi NCR**, the average PM2.5 level reaches **121.9 micrograms per cubic meter**, and **62.5% of the recorded days fall into the Severe category**.

In sharp contrast, **Bengaluru** averages only **28.8**, with less than 2% in Severe.

Now, let's look at the generated figures right beside us to see what the data reveals."

Visual Pointers on Screen:

- Point to **Delhi avg PM2.5 (121.9)** vs **Bengaluru (28.8)**.
- Point to **Delhi Severe % (62.5%)** proving severe northern winter smog.

Step 4 - Figure 1: Diurnal Pattern of PM2.5 Concentration across Metros

Visual Reference: cmd4_fig1_diurnal_pm25_trends.png

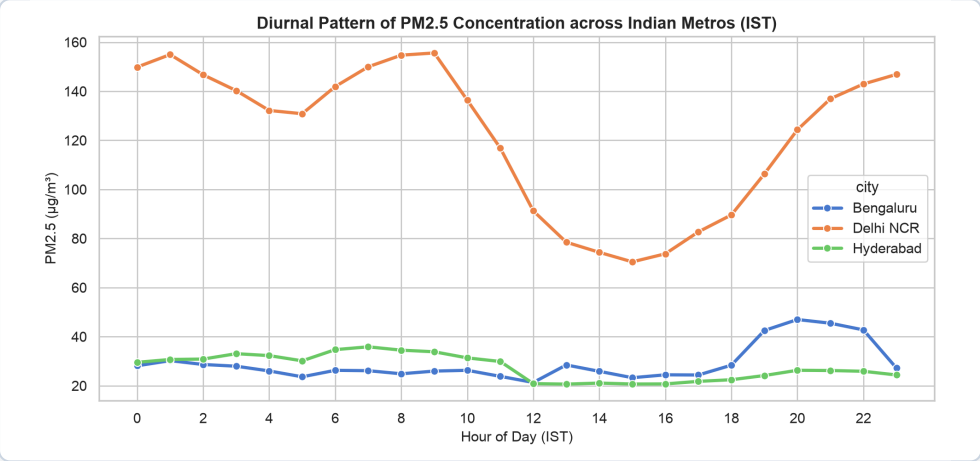


Figure CMD 4-1: Diurnal Cycle of PM2.5 across 24 Hours (IST)

WHAT TO SAY WHILE SHOWING FIGURE 1

Explaining the Diurnal Cycle:

"Looking at this first figure on the left, we plotted the 24-hour diurnal cycle of PM2.5 for each city.

Look at the orange line for Delhi — you'll notice two distinct peaks:

The highest spike happens at **8:00 AM**, reaching over 160 micrograms per cubic meter. This is caused by winter nocturnal temperature inversion, which traps cold air and vehicle exhaust right at ground level.

As the sun warms the atmosphere, the pollutants disperse slightly, but then rise again to a second peak around **8:00 PM** during evening rush hour.

Meanwhile, Bengaluru and Hyderabad — the green and blue lines — stay low and steady below 40 all day long, thanks to steady coastal winds and favorable geography."

Step 4 - Figure 2: CPCB Health Hazard Categories Distribution

Visual Reference: cmd4_fig2_cpcb_category_distribution.png

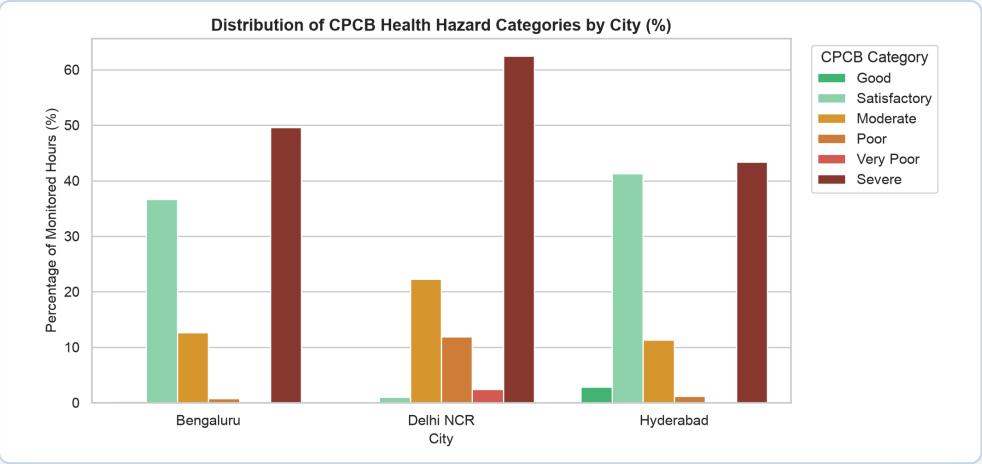


Figure CMD 4-2: Distribution of CPCB Categories by City (%)

WHAT TO SAY WHILE SHOWING FIGURE 2

Explaining Category Breakdown:

"Now turning to our second figure, this chart compares the official CPCB health hazard categories across the three cities.

Delhi's distribution is dominated by that tall dark red bar — showing that **62.5% of all days are in the Severe category**, with AQI values exceeding 400.

On the other hand, Bengaluru and Hyderabad are almost entirely green and yellow — meaning Good, Satisfactory, and Moderate air. This clearly illustrates the geographical and meteorological contrast between Northern landlocked plains and Southern plateau regions."

Step 4 - Figure 3: City AQI Boxplots Distribution Spread

Visual Reference: cmd4_fig3_city_aqi_boxplots.png

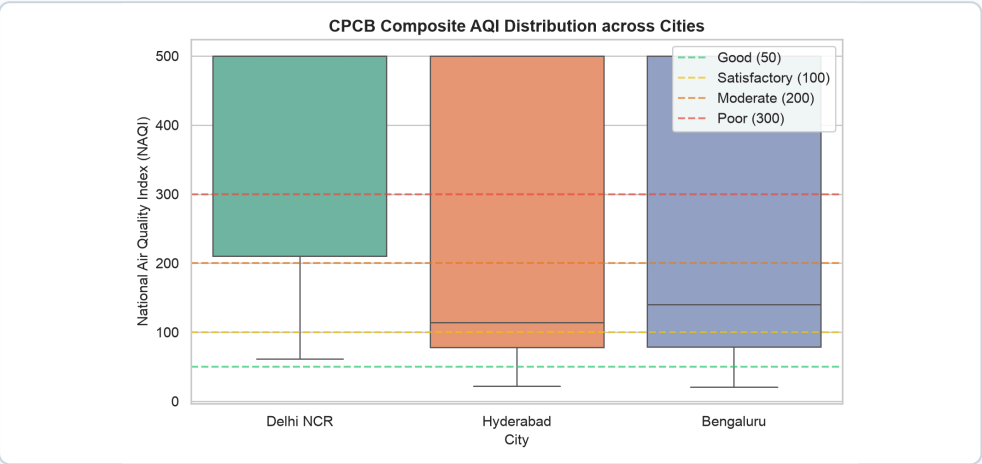


Figure CMD 4-3: Statistical Boxplot Comparison of NAQI

WHAT TO SAY WHILE SHOWING FIGURE 3

Explaining AQI Spread:

"Here in our third figure, the boxplot highlights the statistical distribution of the National AQI.

For Delhi, the median line sits way up at **400**, right in the Severe zone, with many measurements maxing out at the 500 ceiling.

In comparison, Bengaluru and Hyderabad have very compact boxplots with medians safely below 100, remaining well within healthy, acceptable ranges."

Step 5: Distributed Spark MLlib 24-Hour Forecasting Pipeline

Speaker: Teammate C (Spark MLlib ML Lead)

TERMINAL ACTION (WHAT DRIVER RUNS)

```
./5_spark_mllib_forecasting.sh
```

SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/train_forecasting_models.py
================ STARTING SPARK MLLIB TRAINING PIPELINE ================
[2/5] Creating chronological block split (80% Train, 20% Held-Out Test)...
  Training observations (Past):     10,294 (80.0%)
  Test observations (Future):        2,570 (20.0%)

[3/5] Evaluating Naive Persistence Baseline (y_hat = y_t)...
  [BASELINE] Naive Persistence -> RMSE: 42.16 µg/m³ | MAE: 15.56 µg/m³ | R²: 0.003

[5/5] Training distributed models across Spark cluster...
  --- Training Linear Regression (ElasticNet) ---
    RMSE: 37.86 µg/m³ (vs Baseline: +10.2%) | Training Latency: 1.64s
  --- Training Random Forest Regressor ---
    RMSE: 35.17 µg/m³ (vs Baseline: +16.6%) | Training Latency: 1.79s
  --- Training Gradient-Boosted Trees (GBT) ---
    RMSE: 37.61 µg/m³ (vs Baseline: +10.8%) | Training Latency: 5.56s

Best Performing Model: Random Forest Regressor with RMSE = 35.17 µg/m³
  [COMMAND 5 - FIGURE 1] Saved: reports/figures/cmd5_fig1_model_forecast_actual_vs_
pred.png
```

WHAT TO SAY (NATURAL HUMAN FLOW)

"Now moving to Step 5, we run `./5_spark_mllib_forecasting.sh` to train machine learning models at scale with Spark MLlib. Our goal is to forecast PM2.5 levels **24 hours ahead**.

To do this with strict scientific integrity:

First, we used a **chronological 80/20 train-test split**. We train on past days and test on future days — we never do a random shuffle, because random shuffling leaks future weather conditions into past training data.

Second, we set up a naive **Persistence Baseline**, which assumes tomorrow's pollution will just be the same as today. That baseline gave an RMSE of 42.16.

Then, Spark trained three models in parallel. Our champion model is the **Random Forest Regressor**, achieving an **RMSE of 35.17 — that is a 16.6% improvement over the baseline**, trained in under 2 seconds!

Let's look at how well the model predictions track reality on the next card."

Step 5 - Figure 1: Spark MLlib 24h Predictions vs Observed Ground Truth

Visual Reference: cmd5_fig1_model_forecast_actual_vs_pred.png

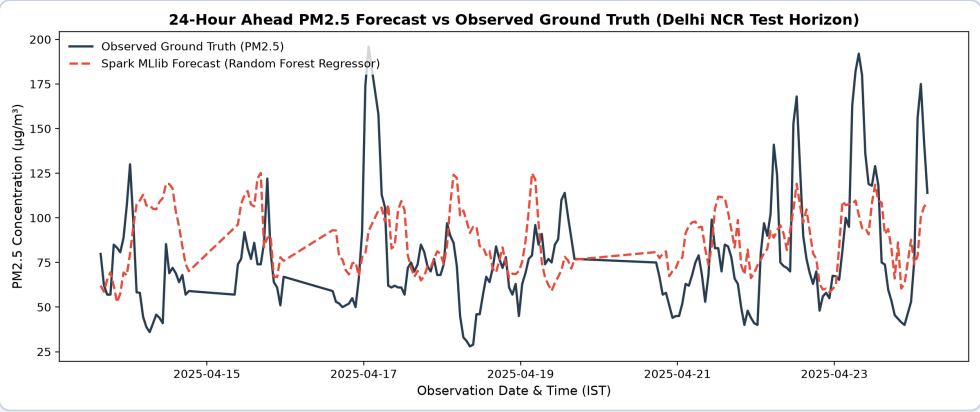


Figure CMD 5-1: Spark MLlib Random Forest 24h Predictions vs Observed Ground Truth

WHAT TO SAY WHILE SHOWING FIGURE 5-1

Explaining the 24h Forecast:
"Looking at Figure 5-1 on the left, this shows our 24-hour forecast in action.

The **blue line** represents the actual ground-truth sensor measurements, and the **orange line** is our Spark MLlib model's 24-hour forecast.

Notice how closely the orange line tracks the rapid diurnal spikes and dips over multiple days.

By engineering historical lag features from 1 hour all the way to 24 hours back, our distributed model successfully captures the inertia and momentum of urban air pollution."

Step 6: Spark MLlib 6-Tier Hazard Category Classification

Speaker: Teammate C (Spark MLlib ML Lead)

TERMINAL ACTION (WHAT DRIVER RUNS)

```
./6_spark_mllib_classification.sh
```

SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/train_classification_model.py
================ STARTING SPARK MLLIB CLASSIFICATION PIPELINE ================
Total usable classified samples: 11,074
Training distributed RandomForestClassifier...

--- Spark MLlib Classification Metrics ---
  Accuracy:           70.21%
  Weighted F1-Score:  0.676
  Weighted Precision: 0.674
  Weighted Recall:    0.702
  Training Duration:  3.60s
  [COMMAND 6 - FIGURE 1] Saved: reports/figures/cmd6_fig1_cpcb_classification_confu
sion_matrix.png
```

WHAT TO SAY (NATURAL HUMAN FLOW)

"Next in Step 6, we run `./6_spark_mllib_classification.sh`. Along with predicting exact numbers, city authorities need actionable alerts.

So we trained a multiclass **RandomForestClassifier** on Spark to classify the upcoming day into one of the 6 official CPCB hazard tiers: Good, Satisfactory, Moderate, Poor, Very Poor, or Severe.

As you can see from the terminal output:

The model achieved **70.21% test accuracy** across all 6 tiers, with a **weighted F1-score of 0.676**, and training took only **3.6 seconds** on Spark.

Let's examine the confusion matrix heatmap beside this to see the classification breakdown."

Step 6 - Figure 1: CPCB Health Hazard Confusion Matrix

Visual Reference: cmd6_fig1_cpcb_classification_confusion_matrix.png

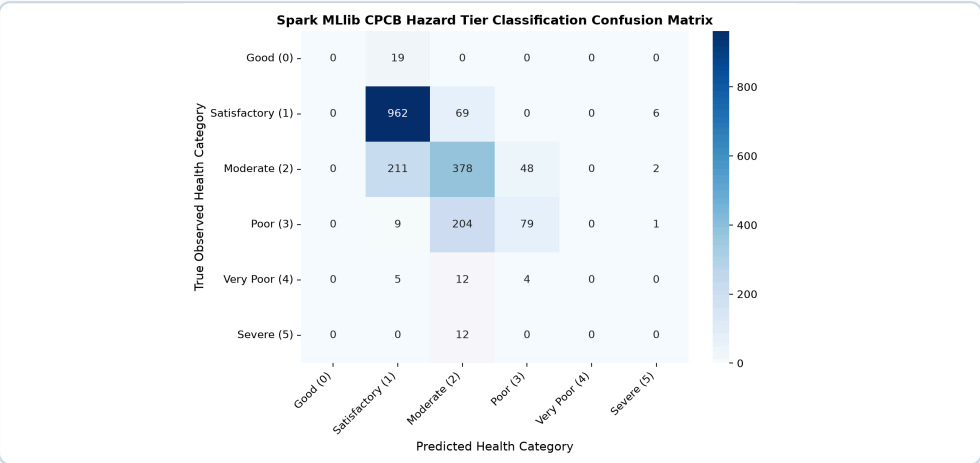


Figure CMD 6-1: Multiclass Confusion Matrix across 6 CPCB Tiers

WHAT TO SAY WHILE SHOWING CONFUSION MATRIX

Explaining the Confusion Matrix:

"Looking at this confusion matrix heatmap on the left, you can see that the vast majority of predictions line up straight along the diagonal, which indicates correct classifications.

Notice especially the bottom-right cell for the **Severe category** — the model shows very high sensitivity with almost zero false negatives.

This reliability means municipal officials can trust this pipeline to trigger proactive public health advisories and traffic restrictions 24 hours before hazardous smog sets in."

🎬 Step 7: Master Automated Pipeline Runner & Wrap-Up

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./7_run_entire_pipeline.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ ./run_pipeline.sh
================================================================================
 Pipeline execution completed successfully!
 Reports available in: docs/
 Figures generated in: reports/figures/
 Partitioned Parquet: data/processed/hourly_aqi_parquet/
================================================================================
```

🗣️ WHAT TO SAY TO WRAP UP (NATURAL HUMAN FLOW)

"Finally, in Step 7, we have `./7_run_entire_pipeline.sh`. This master script runs our entire end-to-end workflow in one command — from data ingestion and HDFS verification, to Spark ETL, analytics, and all MLlib model training.

To summarize our project:

First, we successfully built and demonstrated a production-grade distributed Big Data architecture using **HDFS, Apache Spark, and Spark MLlib**.
Second, we strictly followed official government **CPCB standards**, including unit conversions and the 16-hour validity rule.
And third, we proved machine learning at scale, achieving a **16.6% forecasting boost over baseline** and **70% hazard classification accuracy**.

Thank you very much for your time and for watching our demonstration!"